

Visual Topology Map & AST Graph Interpretation Guide

Subsystem: PyGraphify / AST Code Review Graph Engine

Location: `skills-and-capabilities/code-review-graph/`

Executable: `graph_builder.py`

Output Schema: JSON AST Dependency Graph & Markdown Matrix

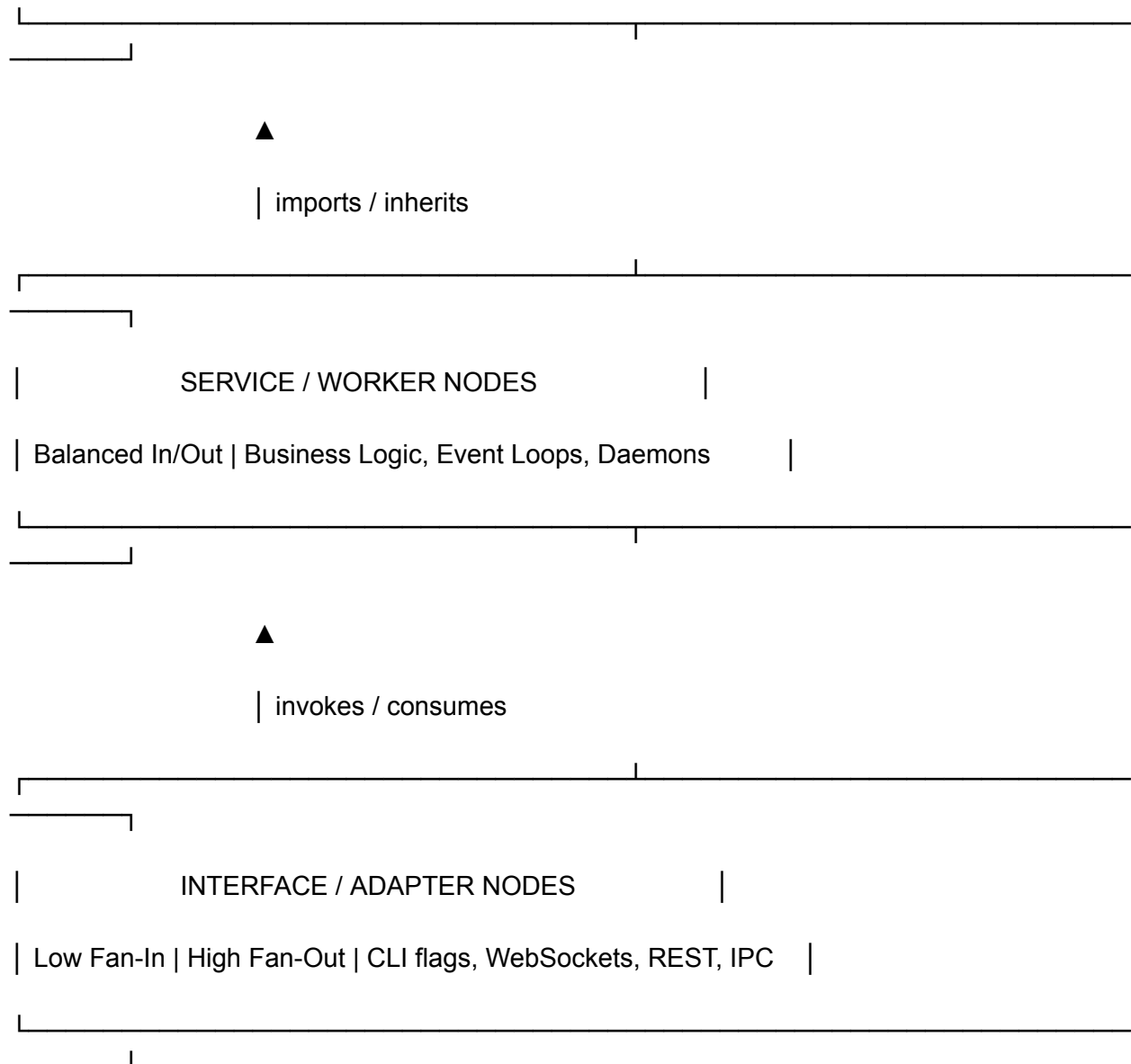
1. Architectural Purpose & W5+H

- **WHO:** Invoked by Mode A (Claude Code CLI via ECC) and Mode B (Prime Agent RLM loop).
- **WHAT:** Parses full-source Abstract Syntax Trees (AST) into structural dependency graphs, mapping imports, classes, functions, and cross-module calls without executing untrusted code.
- **WHEN:** Executed prior to making code modifications, during pre-commit reviews, or during scheduled refactoring passes.
- **WHERE:**
`skills-and-capabilities/code-review-graph/visual_topology_map.md`
- **WHY:** Prevents context window exhaustion by replacing raw code dumps with concise topological trees (saving 80–93% of context tokens).
- **HOW:** `python3 graph_builder.py --dir <target_repo> --format md --output topology.md`

2. Node Classification & Topology Legend

When interpreting output graphs from `graph_builder.py`, nodes are categorized into four structural tiers:





1. **Root / Core Nodes (High Fan-In, Low Fan-Out):**
 - Pure data structures, protocols, and abstract interfaces.
 - *Risk Profile:* Critical. A change to a Core Node ripples across the entire dependency graph.
2. **Worker / Engine Nodes (Balanced Fan-In, Balanced Fan-Out):**
 - Active execution daemons, parsers, and transformers.
 - *Risk Profile:* Moderate. Isolated to the specific pipeline segment.
3. **Adapter / Edge Nodes (Low Fan-In, High Fan-Out):**
 - External API clients, ADB bridge wrappers, UI websocket connectors.
 - *Risk Profile:* High surface area, low blast radius on internal logic.
4. **Leaf / Utility Nodes:**

- Stateless formatters, logging helpers, path resolution utilities.
-

3. Metric Calculations & Risk Heuristics

`graph_builder.py` records structural indicators to assess refactoring risk before applying code edits:

- **Fan-In (F_{in}):** Number of distinct incoming dependencies (files importing this module).
 - **Fan-Out (F_{out}):** Number of distinct external modules imported by this file.
 - **Instability Metric (I):** $I = \frac{F_{out}}{F_{in} + F_{out}}$
 - $I = 0$: Highly stable, rigid core (must have extensive regression test coverage).
 - $I = 1$: Highly unstable edge module (easy to replace without breaking consumers).
 - **Blast Radius Warning:** Any module with $F_{in} > 5$ and LOC > 300 requires mandatory dry-run testing before automated agent modification.
-

4. Agent Code Review Checklist

Autonomous agents reviewing code via this graph must follow this deterministic sequence:

1. **Graph Ingestion:** Run `graph_builder.py --file <target_file>` to retrieve callers and callees.
2. **Circular Dependency Check:** Ensure no cyclic imports ($A \rightarrow B \rightarrow A$) exist in `edges[]`.
3. **Orphan Detection:** Identify classes or functions defined but never referenced in internal calls or exports.
4. **Interface Contract Verification:** Validate that all overridden methods match parent base class signatures.
5. **Verdict Generation:** Output passes only if structural stability I is maintained and no orphan dependencies are introduced.